

§11.4.108 Four-Layer Verification

“ Session Commits

457cca4..fab6707

Revision: 1 Last modified: 2026-08-18T20:50:00Z

Scope

Verifies the ~28 commits landed in this session (457cca4 -> fab6707, 81 files changed) against the running boba stack, per §11.4.108 (SOURCE -> ARTIFACT -> RUNTIME-ON-CLEAN-TARGET -> USER-VISIBLE).

Phase 1 “ Runtime-impact categorization

git diff --name-only 457cca4..fab6707 enumerated. Every changed file classified:

Category	Files	Runtime impact
C “ rebuild required	qBitTorrent-go/internal/jackettapi/health.go, health_test.go	YES “ boba-jackett is build: context: ./qBitTorrent-go, dockerfile: Dockerfile.jackett in docker-compose.yml (not a bind-mount)
B “ host-level, non-container	scripts/install-resource-pressure-timer.sh + scripts/systemd/user/boba-resource-pressure-check.{service,timer}	NO “ systemd --user timer, entirely outside the docker-compose stack
A “ no runtime impact	Everything else: docs/**, challenges/**, scripts/commit-push-all.sh, scripts/install-dev-tools.sh, scripts/capture-workable-items-db-delta.sh, scripts/pre_build_verification.sh, scripts/host-power-management/check-no-suspend-calls.sh, constitution (submodule pointer)	NO “ dev tooling / challenge scripts / docs / governance, none container-mounted or container-built

Verdict: exactly one runtime-affecting source pair (BOB-112's TTL-cache fix in health.go). No docker-compose.yml, no Dockerfile.jackett, no download-proxy/src/, no plugins/, no config/ changes this session.

Phase 2 " Rebuild + reflash

Finding before acting: the *already-running* boba-jackett container's image (962f4452c725...) was built at 2026-08-18T19:06:17Z " after health.go's last touching commit 91b52db (19:02:50Z) " because a prior subagent in this same session (commit e2a2e3e, evidence at docs/qa/BOB-112/) had already rebuilt + RED/GREEN-mutation-verified this exact fix with live wrk load tests (97.1%→0.0% timeout rate, full methodology + numbers in docs/qa/BOB-112/summary.md). Confirmed no commit after 91b52db touched health.go (git log 91b52db..fab6707 -- health.go -> empty).

This task's job was therefore to **redeploy on a genuinely clean target** (Â§11.4.108 clause 4 " eliminate the stale-deployment/shadow layer by construction) and confirm the fix holds, rather than repeat the prior subagent's RED/GREEN mutation study.

Ran ./start.sh --recreate (the CLAUDE.md-sanctioned Level-3 restart for "docker-compose.yml, start-proxy.sh, env var, base image" changes " the correct level given a rebuilt image needed a fresh container instance). Full transcript: start_sh_recreate.log.

- Tracker-cookie loader ran first (per operator mandate 2026-08-15): all 5 trackers (rutracker/nnmclub/rutor/kinozal/iptorrents) reported UNCHANGED (idempotent) " no drift, no re-extraction needed.
- boba-ctl (Go orchestrator) executed podman-compose ... down then ... up -d " exit 0 both steps.
- Only boba's own 4 services (qbittorrent, jackett, qbittorrent-proxy, boba-jackett) were touched. Confirmed via before/after podman ps -a (container_status.log) that the 15 unrelated containers belonging to other projects on this host (proxy-*, lava-*, helix-*, deploy_caddy_1, wonderful_dijkstra) were **not restarted, not recreated, not touched** " their Up 2 hours / Up 12 hours uptimes are unchanged across the before/after snapshots.
- All 4 boba containers reached healthy within ~1 minute (qbittorrent/jackett: healthy at 52s; boba-jackett/qbittorrent-proxy: healthy at 46s).
- Post-recreate cookie lengths (byte counts only, Â§11.4.10 " no values logged) are **byte-identical** to the pre-recreate baseline for all 5 trackers (cookie_state_post_recreate.log).

Phase 3 " Runtime verification

All 5 endpoints reachable post-recreate (endpoint_health.log, verbatim):

Endpoint Result

GET :7186/
api/v2/
app/
version 200, body v5.2.3
(qBittorrent
via proxy)

POST
:7186/api/
v2/auth/
login 204 (qBittorrent's own success response)
(admin/
admin)

GET :7187/
health 200, {"status":"healthy","service":"merge-search","version":"1.0"
(merge
service)

GET :7189/
healthz 200,
(boba- {"status":"ok","db_ok":true,"jackett_ok":true,"version":"0.1.0"
jackett,
NEW TTL
cache)

GET :9117/
api/v2.0/
server/
config 302 (redirect to admin auth " reachable, expected for an unauthenticated
(Jackett
upstream)

BOB-112 fix " live verification on the fresh container

Artifact layer (artifact_layer_check.txt): the BOB-112 fix's unique log.Printf format string (added in 91b52db, absent in every prior version of health.go) is present in the **freshly-recreated** container's binary:

```
boba-jackett: /healthz Jackett cache refresh #%d (hits=%d misses=%d ratio=
```

Image digest 962f4452c725... " identical to the digest the prior subagent's RED/GREEN mutation study (docs/qa/BOB-112/summary.md) recorded as the confirmed post-fix, post-revert-verification build. No drift.

Runtime layer (wrk_healthz_evidence.log): `brief wrk -t2 -c50 -d5s --timeout 3s --latency http://localhost:7189/healthz` against the container that had been up for well under a minute (genuinely cold cache at test start):

```
270085 requests in 5.00s, 48.17MB read
Requests/sec: 54008.16
Latency: avg 1.08ms, p50 844us, p90 2.31ms, p99 4.42ms
```

0 timeouts / 0 socket errors (wrk prints no “Socket errors” line when all three counters “connect/read/write/timeout” are zero) “ well under the task’s <5% threshold, and consistent with the prior subagent’s full 30sx100c study (0.0% timeouts, 27k req/s).

Container logs (podman logs boba-jackett) show exactly 2 upstream Jackett.GetCatalog() calls across the whole test window “ one at boot (cache refresh #1, cold cache) and one ~31s later once the 30s TTL first expired “ confirming the cache, not luck, is what collapsed 270k /healthz hits into 2 real upstream calls.

Honest gaps / not covered by this task

- **No image rebuild was forced.** ./start.sh --recreate is compose down && compose up -d “ it does not pass --build, so it reused the existing boba_boba-jackett:latest image rather than compiling a fresh one from current HEAD (fab6707). This is not a gap for BOB-112 specifically (proven byte-identical via the format-string + digest match above, and git log 91b52db..fab6707 -- health.go is empty so there is nothing newer to rebuild), but it means this task did not independently *reproduce* a build from source “ it verified the artifact already known to be correct survives a clean container-level redeploy. A from-scratch podman-compose build boba-jackett was not run because it would have produced a byte-identical image to the one already running (no source delta since the prior subagent’s own build), and CLAUDE.md’s Critical Constraints table has no sanctioned start.sh subcommand for “force-rebuild a container’s Go source without a compose/Dockerfile/env change” (only --reload-python, --reload-plugins, and --recreate exist) “ using a raw podman-compose build would have stepped outside the orchestrator-only mandate for a rebuild this task’s own evidence shows was unnecessary.
- **Â§11.4.185 manual QA (human confirmation) not performed** “ out of scope for this automated verification subagent; this report is the automated Â§11.4.108 evidence layer only, not the terminal human sufficiency gate.
- **qbittorrent-proxy (Python/FastAPI merge service) source unchanged this session** “ verified reachable (:7187/health 200) but not independently re-tested beyond the health endpoint, since no commit touched download-proxy/src/.
- The host-level boba-resource-pressure-check.timer (BOB-076/task-77) was spot-checked (systemctl --user list-timers, active, fired ~5 min prior) for completeness but is **not part of the boba container stack** and its own evidence trail already lives at docs/qa/task-77/.

Verdict

PASS “ the one runtime-affecting change this session (BOB-112 TTL cache) is confirmed live on a genuinely fresh container instantiation via both

artifact-layer (binary string + image digest match) and runtime-layer (0%-timeout wrk run + exact cache-refresh-count log evidence) proof. All 5 boba-stack endpoints reachable post-recreate. Zero unrelated host containers disturbed. All 5 tracker cookies preserved byte-length-identical across the recreate.